Exercise 1. Inventory Management System

1. Data Structures are essential in handling large inventories because a huge amount of product data needs to be stored, managed, and retrieved efficiently. Appropriate data structures help reduce the time required for operations such as searching, updating, and deleting products, while also optimizing memory usage and improving the overall performance of the inventory management system.

   Since each product has a unique ***Product ID***, HashMap is the most suitable data structure for this inventory management system. It stores data in a key-value pair format, where the Product ID acts as the key and the Product object acts as the value. This allows operations such as insertion, searching, updating, and deletion to be performed in O(1) average time complexity. As a result, product records can be accessed directly without traversing the entire collection, making the system faster and more efficient for handling large inventories.

2. Implementation:

```
import java.util.HashMap;

//Product Class

class Product{
    int productID;
    String productName;
    int quantity;
    int price;

    public Product(int productID, String productName, int quantity, int
price){
        this.productID = productID;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }
}
```

```
class InventoryManagementSystem{
    HashMap<Integer, Product> products;
    public InventoryManagementSystem(){
        products = new HashMap<>();
    }
public void addProduct(String name, int id, int quantity, int price){
        products.put(id, new Product(id, name, quantity, price));
    }

    public void updateProduct(int id, int quantity, int price){
        if(products.containsKey(id)){
            Product product = products.get(id);
            product.quantity = quantity;
            product.price = price;
        }
    }
    public void deleteProduct(int id){
        products.remove(id);
    }
}
```

Analysis:

Operation	Complexity
Add	O(1)
Update	O(1)
Delete	O(1)

Optimization:

Since HashMap already provides O(1) average time complexity for insertion, searching, updating, and deletion, the system is highly efficient. Further optimization can be achieved by initializing the HashMap with an appropriate capacity if the expected number of products is known in advance. This reduces the need for rehashing and improves performance. Additionally, duplicate Product IDs can be prevented by checking whether a key already exists before adding a new product